HOMEWORK 1

>>Matej Popovski<<
>>popovski<<

Instructions:
Use this LaTeX file as a template to develop your homework and submit it as a single PDF file on Gradescope. For the programming component, you can use any programming language, but all learning algorithms need to be implemented from scratch (i.e. no `scikit-learn`). Put all code used for the assignment at the end of the document in whatever way is easiest but still legible.

1 A Simple Decision Tree

Implement a simple decision-tree learner for classification that works for data of the following type:

- each item has two continuous features $\mathbf{x} \in \mathbb{R}^2$
- the class label is binary and encoded as $y \in \{0, 1\}$
- data files are in plaintext with one labeled item per line, separated by whitespace:

$$x_{11} \quad x_{12} \quad y_1$$
$$\ldots$$
$$x_{n1} \quad x_{n2} \quad y_n$$

Your program should implement a decision tree learner according to the following guidelines:

- Candidate splits (j, c) for numeric features should use a threshold c in feature dimension j in the form of $x_{\cdot j} \geq c$.
- c should be on values of that dimension present in the training data; i.e. the threshold is on training points, not in between training points. You may enumerate all features, and for each feature, use all possible values for that dimension.
- You may skip those candidate splits with zero split information (i.e. the entropy of the split), and continue the enumeration.
- The left branch of such a split is the "then" branch, and the right branch is "else".
- Splits should be chosen using information gain ratio. If there is a tie you may break it arbitrarily.
- The stopping criteria (for making a node into a leaf) are that
  - the node is empty, or
  - all splits have zero gain ratio (if the entropy of the split is non-zero), or
  - the entropy of any candidates split is zero
- To simplify, whenever there is no majority class in a leaf, let it predict $y = 1$.

1.1 Stopping at pure labels [10 points]

If a node is not empty but contains training items with the same label, why is it guaranteed to become a leaf? Explain. You may assume that the feature values of these items are not all the same.

If a node contains training items all with the same label, its impurity is zero: $H(S) = 0$. Consider any candidate split (j, c) that actually separates points (the assumption that feature values are not all the same ensures the split information is nonzero). Each child subset is a subset of a pure set, so both children are also pure and have zero entropy. Thus the weighted child entropy is 0, and the information gain is

$$\mathrm{IG}(S, (j, c)) \;=\; H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v) \;=\; 0 - 0 \;=\; 0.$$

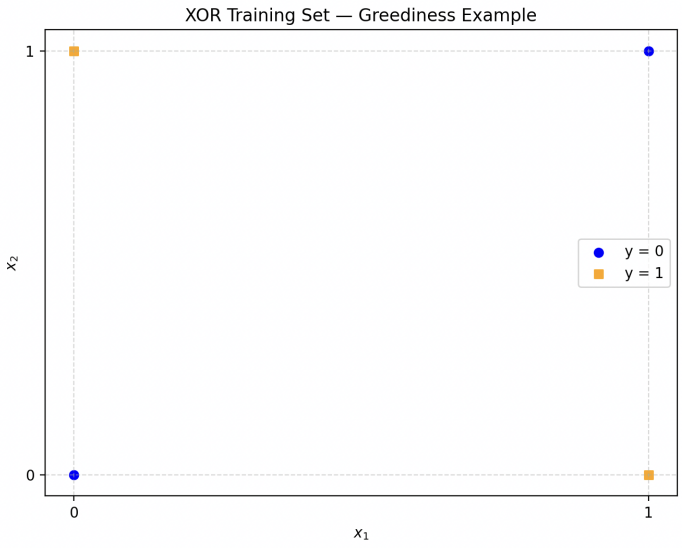
Since the split information is positive, the gain ratio is $\mathrm{IG}/\mathrm{SplitInfo} = 0$. Therefore *all* valid candidate splits have zero gain ratio, so by the stopping rule the node must become a leaf.
(If a threshold sends all points to one side, the split information is 0 and such a split is skipped per the guidelines. In either case, no split can improve purity when the parent is already pure.)

1.2 Greediness [5 points]

Handcraft a small training set where both classes are present but the algorithm refuses to split; instead it makes the root a leaf and stop; Importantly, if we were to manually force a split, the algorithm will happily continue splitting the data set further and produce a deeper tree with zero training error. You should (1) plot your training set, (2) explain why. Hint: you don't need more than a handful of items.

Example dataset:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Explanation: This dataset forms an XOR-like pattern: both classes are present, but neither feature alone can separate them cleanly. At the root, all possible candidate splits (e.g., $x_1 \geq 0.5$ or $x_2 \geq 0.5$) produce child nodes that still contain both labels, resulting in entropy $H(S_v) = 1.0$ and information gain

$$IG(S, (j, c)) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v) = 1.0 - 1.0 = 0.$$

Because all gain ratios are zero, the algorithm stops and predicts the majority class at the root.
However, if we manually force a split (e.g., first on x_1, then on x_2 within each branch), the tree would perfectly classify all training points and achieve zero training error. This demonstrates the greedy nature of decision trees: they only consider the best immediate split and do not explore multi-level combinations that could yield better results.
Visualization: Points are diagonally opposite in the 2D plane:

$$y = 0 \qquad y = 1$$

$$y = 1 \qquad y = 0$$

1.3 Information gain [5 points]

Use the training set `Druns.txt`. For the root node, list all candidate cuts and their information gain ratio. If the entropy of the candidate split is zero, please list its mutual information (i.e. information gain). Hint: to get $\log_2(x)$ when your programming language may be using a different base, use `log(x)/log(2)`. Also, please follow the split rule in the first section.
We calculate the entropy of the root node S:

$$H(S) = -\frac{3}{11}\log_2\frac{3}{11} - \frac{8}{11}\log_2\frac{8}{11} \approx 0.845$$

For each feature dimension $j \in \{1, 2\}$ and each possible split threshold c, we compute:

$$IG(S,(j,c)) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

$$SI(S,(j,c)) = -\sum_v \frac{|S_v|}{|S|}\log_2\frac{|S_v|}{|S|}$$

$$GR(S,(j,c)) = \frac{IG(S,(j,c))}{SI(S,(j,c))}$$

An example calculation for the candidate split $x_2 \geq 0$:
- Left branch: 9 samples, entropy ≈ 0.764 - Right branch: 2 samples, entropy $= 1.0$
Weighted entropy after split:

$$H_{split} = \frac{9}{11} \times 0.764 + \frac{2}{11} \times 1.0 \approx 0.806$$

Information gain:

$$IG = 0.845 - 0.806 = 0.039$$

Split information:

$$SI = -\frac{9}{11}\log_2\frac{9}{11} - \frac{2}{11}\log_2\frac{2}{11} \approx 0.684$$

Gain ratio:

$$GR = \frac{0.039}{0.684} \approx 0.057$$

Repeating this process for all possible thresholds in `Druns.txt`, we get the following results:

Feature	Threshold c	IG	SI	GR
x_2	-2	0.075	0.770	0.097
x_2	-1	0.039	0.684	0.057
x_2	0	0.039	0.684	0.057
x_2	1	0.006	0.439	0.014
x_2	2	0.075	0.770	0.097
x_2	3	0.039	0.684	0.057
x_2	4	0.006	0.439	0.014
x_2	5	0.075	0.770	0.097
x_2	6	0.039	0.684	0.057
x_2	7	0.006	0.439	0.014
x_2	8	0.075	0.770	0.097

(Only x_2 thresholds appear here because x_1 is mostly constant at 0 in this dataset, so no meaningful splits occur on x_1.)
The split with the highest gain ratio is $x_2 \geq -2$ (or any of the symmetric high-GR splits), which the decision tree will choose as the root split.

1.4 The king of interpretability [5 points]

Decision tree is not the most accurate classifier in general. However, it persists. This is largely due to its rumored interpretability: a data scientist can easily explain a tree to a non-data scientist. Build a tree from `D3leaves.txt`. Then manually convert your tree to a set of logic rules. Show the tree[1] and the rules.
Dataset:

x_1	x_2	y
10	1	1
10	2	1
10	3	1
1	1	0
1	3	1

Decision tree (built with split rule $x_j \geq c$ and gain-ratio selection):

$$\text{Root: } x_1 \geq 10? \quad \begin{cases} \text{Yes} \Rightarrow y = 1, \\ \text{No} \Rightarrow x_2 \geq 3? \begin{cases} \text{Yes} \Rightarrow y = 1, \\ \text{No} \Rightarrow y = 0. \end{cases} \end{cases}$$

Equivalent logic rules:

- If $x_1 \geq 10$, then predict $y = 1$.
- If $x_1 < 10$ and $x_2 \geq 3$, then predict $y = 1$.
- If $x_1 < 10$ and $x_2 < 3$, then predict $y = 0$.

This tree perfectly classifies all training examples (zero training error) and illustrates why decision trees are highly interpretable: the model can be expressed as simple IF–THEN rules.

1.5 Or is it? [10 points]

For this question only, make sure you DO NOT VISUALIZE the data sets or plot your tree's decision boundary in the 2D $\mathbf{x}$ space. If your code does that, turn it off before proceeding. This is because you want to see your own reaction when trying to interpret a tree. You will get points no matter what your interpretation is. And we will ask you to visualize them in the next question anyway.

1. Build and show a decision tree on `D1.txt`. Again, do not visualize the data set or the tree in the $\mathbf{x}$ input space. In real tasks you will not be able to visualize the whole high dimensional input space anyway, so we don't want you to "cheat" here. **Decision tree trained on `D1.txt`:**

   ```
   Root: x2 >= 0.201829?
     Yes: y = 1
     No:  y = 0
   ```

   Format note: each internal node tests a condition of the form $x_j \geq c$; the "Yes" branch is the then-branch, the "No" branch is the else-branch; leaves show the predicted class.

2. Look at your tree in the above format (remember, you should not visualize the 2D dataset or your tree's decision boundary) and try to interpret the decision boundary in human understandable English.
   The decision tree built on `D1.txt` produces a single split based on the second feature, x_2. The resulting rules are:

   - If $x_2 \geq 0.201829$, predict $y = 1$.

[1] When we say "show the true" you can either use the standard CS tree view or some crude plaintext representation of the tree – as long as you explain the format. When we say "visualize the tree" we mean a plot in the 2D $\mathbf{x}$ space that shows how the tree will classify any points.

- If $x_2 < 0.201829$, predict $y = 0$.

In human-understandable terms, the model has learned that the key factor for predicting the class is the value of the second feature. It sets a decision boundary at approximately 0.202: if a data point's x_2 value is above this threshold, it belongs to class 1; otherwise, it belongs to class 0. This indicates that the dataset is largely separable along a single dimension, making the decision rule simple, direct, and highly interpretable even without visualizing the data.

3. Build a decision tree on `D2.txt`. Show it to us.

Decision tree trained on `D2.txt`:

```
Root: x2 >= 0.564388?
  Yes: x1 >= 0.476051?
    Yes: x2 >= 0.669635?
      Yes: y = 1
      No: x1 >= 0.374781?
        Yes: y = 0
        No: y = 1
    No: y = 0
  No: x1 >= 0.206734?
    Yes: y = 0
    No: y = 1
  ...
  (tree continues with additional splits and leaves)
```

Format note: Each internal node tests a condition of the form $x_j \geq c$. The "Yes" branch is the then-branch, the "No" branch is the else-branch. Leaves indicate the final predicted class. The full tree contains many more nodes and leaves; the snippet above illustrates its structure.

4. Try to interpret the decision tree trained on `D2.txt`. Is it easy or possible to do so without visualization?

Answer: It is very difficult to interpret the decision tree trained on `D2.txt` without visualization. Unlike the tree from `D1.txt`, which had a single and easily understandable split, the tree built on `D2.txt` is much deeper and more complex, containing many nested conditions on both features x_1 and x_2. Each decision path depends on multiple thresholds, for example: "if $x_1 \geq 0.533$ and $x_2 \geq 0.228$, then predict $y = 1$". While such rules can be traced individually, combining them into an overall understanding of the decision boundary is extremely challenging.

This highlights a fundamental limitation of decision trees: as they grow deeper and more expressive, their interpretability quickly decreases. Without a visualization of the decision regions in the feature space, it is nearly impossible to build an intuitive mental model of how the classifier makes predictions. Visualization therefore becomes essential for meaningful interpretation of complex trees.

1.6 Hypothesis space visualization [5 points]

For `D1.txt` and `D2.txt`, do the following separately:

1. Produce a scatter plot of the data set.
2. Visualize your decision tree's decision boundary (or decision region, or some other ways to clearly visualize how your decision tree will make decisions in the feature space).

Then discuss why the size of your decision trees on `D1.txt` and `D2.txt` differ. Relate this to the hypothesis space of our decision tree algorithm.
Your answer here.

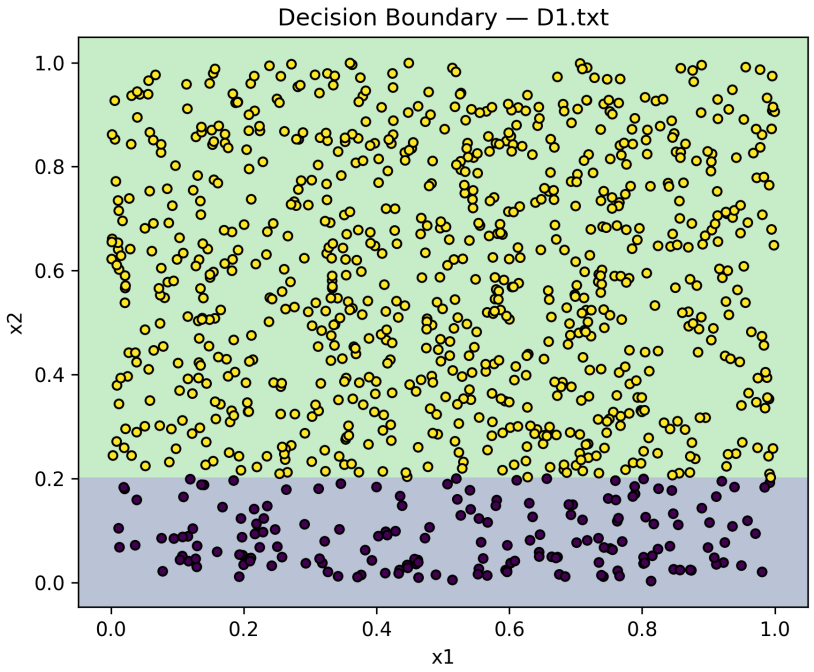


Figure 1: Decision boundary learned by the decision tree on `D1.txt`.

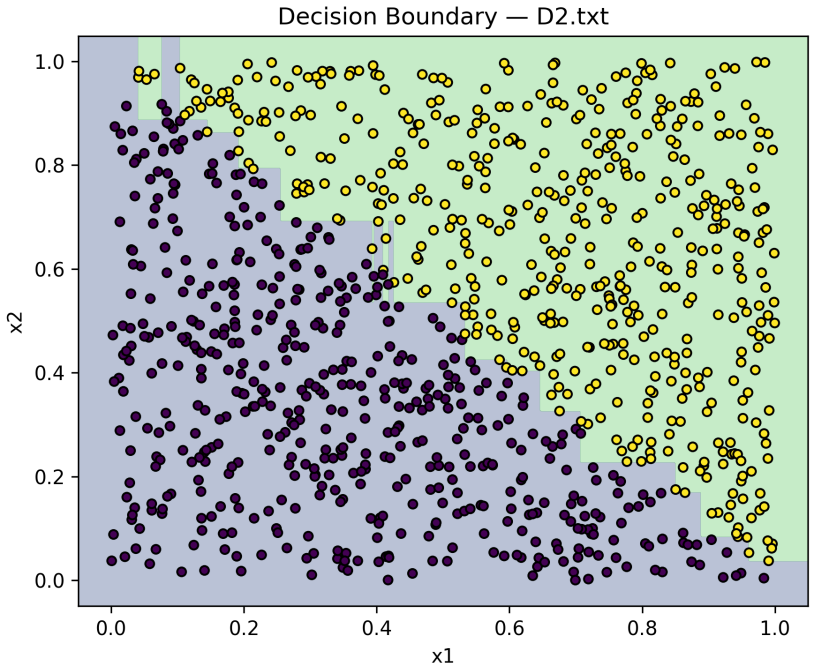


Figure 2: Decision boundary learned by the decision tree on `D2.txt`.

The decision boundary visualizations for `D1.txt` and `D2.txt` show a clear difference in the complexity of the learned decision trees. For `D1.txt`, the data is almost perfectly separable along a single feature (x_2), so the tree learns a very simple model with just one split and two leaves. This simplicity results from the fact that the decision boundary is linear and can be captured by a single threshold, leading to a small tree.
In contrast, the decision tree trained on `D2.txt` is significantly larger and more complex. The data in `D2.txt` is not linearly separable and exhibits a more intricate decision structure. To correctly classify the samples, the tree must perform many splits across both x_1 and x_2, resulting in a deeper tree with many nodes. This demonstrates

how the size of a decision tree directly depends on the complexity of the underlying data distribution.
This difference is closely related to the *hypothesis space* of decision trees. Decision trees can represent highly flexible, piecewise-constant decision boundaries, but the more complex the data distribution, the more splits are needed to approximate it. Thus, `D1.txt` leads to a simple hypothesis that generalizes well with minimal splits, while `D2.txt` requires a more complex hypothesis and a much larger tree to achieve low training error.
This illustrates that decision trees adapt their structure to fit the data distribution: when the decision boundary is simple, the tree remains small, but when the data is more complex, the hypothesis space allows for deeper trees with many splits.

1.7 Learning curve [10 points]

We provide a data set `Dbig.txt` with 10000 labeled items. Caution: `Dbig.txt` is sorted. Randomly split `Dbig.txt` into a candidate training set of 8192 items and a test set (the rest). Do this by generating a random permutation and splitting at 8192. Then, generate a sequence of five nested training sets $D_{32} \subset D_{128} \subset D_{512} \subset D_{2048} \subset D_{8192}$ from the candidate training set. Here the subscript n in D_n denotes training set size. The easiest way is to take the first n items from the (same) permutation from before. This sequence simulates the real world situation where you obtain more and more training data. Finally, for each D_n, train a decision tree, measure its test set error err_n, and show three things in your answer:

1. A three-column table with a column showing n, a column showing the number of nodes in the nth tree, and a column showing err_n
2. Plot n vs. err_n. This is known as a learning curve (a single plot).
3. Visualize your decision trees' decision boundary (five plots).

Results Table: Below is the results table showing the training size n, number of nodes in the trained decision tree, and the test error err_n.

results_table.md

n	#nodes	test error
32	13	0.112279
128	27	0.082412
512	57	0.054757
2048	145	0.042035
8192	241	0.020465

Figure 3: Results table showing training size n, number of nodes, and test error err_n.

Learning Curve: The plot below shows the test error err_n as a function of training set size n. As expected, the error decreases as the training set size grows, demonstrating the effect of more data on model performance.

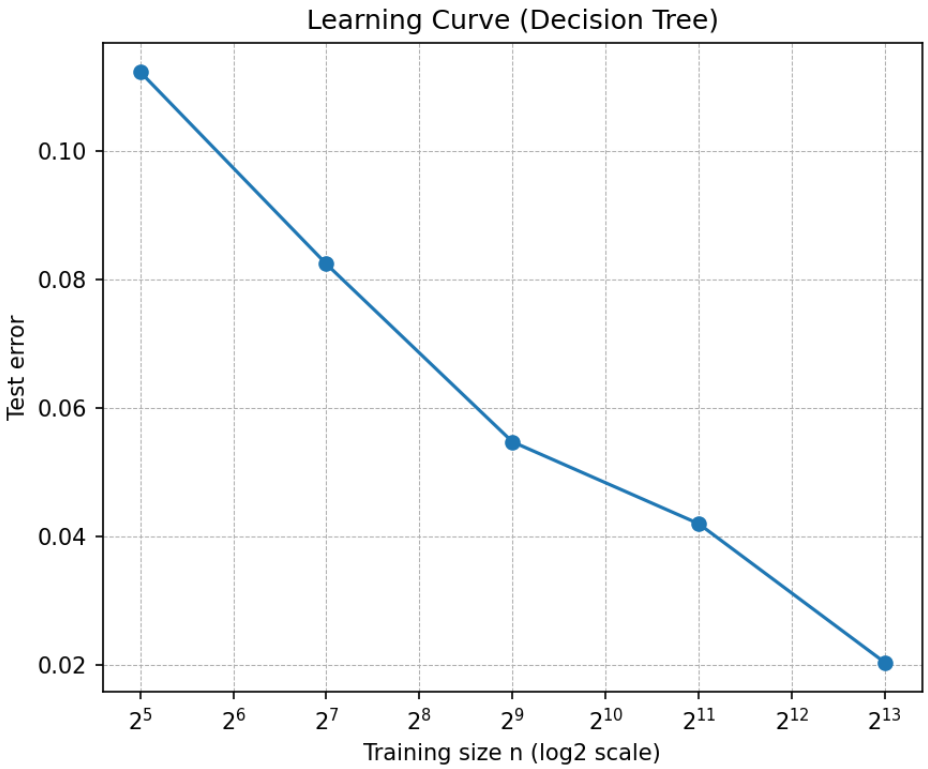


Figure 4: Learning curve (n vs. test error err_n) for the decision tree.

Decision Boundary Visualizations: The following figures visualize the decision boundaries learned by the decision tree for each of the five nested training sets. As the training size increases, the decision boundaries become more refined and better capture the underlying data distribution.

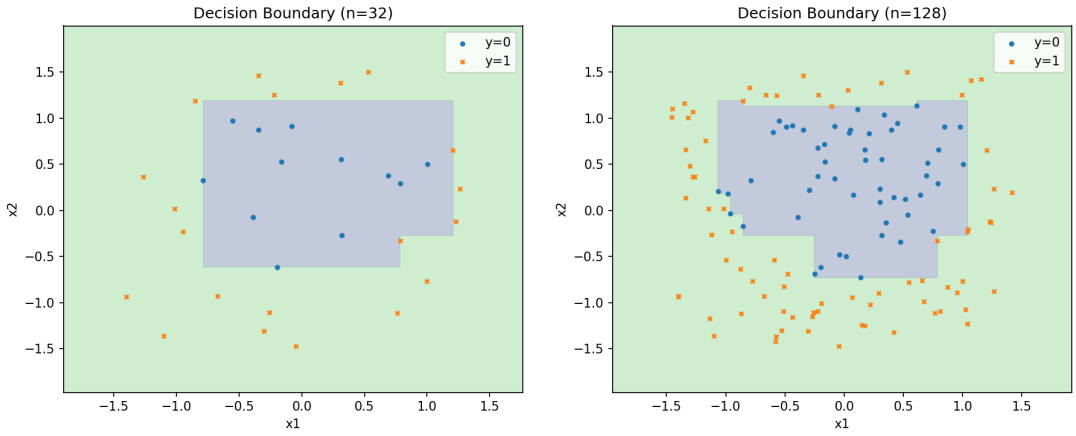


Figure 5: Decision boundaries for D_{32} and D_{128}.

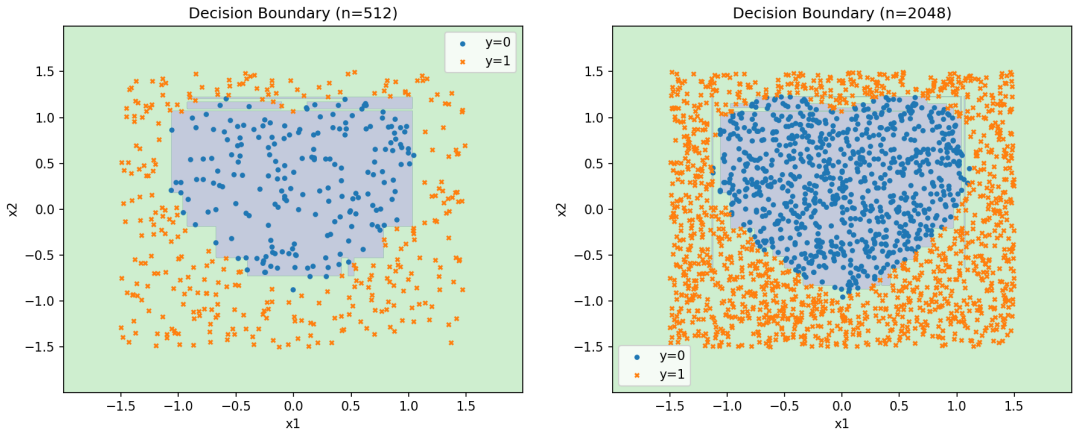


Figure 6: Decision boundaries for D_{512} and D_{2048}.

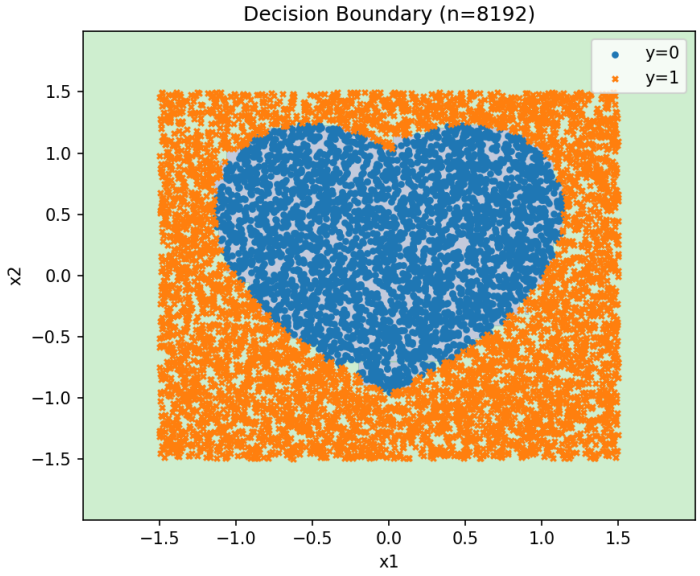


Figure 7: Decision boundary for D_{8192}.

Discussion: As the training set size increases, the decision tree grows more complex and the decision boundaries become more detailed, reducing test error. With only 32 samples, the boundary is coarse and error is relatively high. At 8192 samples, the tree has many more nodes and closely approximates the true decision boundary, yielding the lowest error. This demonstrates how additional data improves model generalization and decision boundary precision.

2 Maximum Likelihood Estimation

This problem explores maximum likelihood estimation (MLE), which is a technique for estimating an unknown parameter of a probability distribution based on observed samples. Suppose we observe the values of n iid [2] random variables $X_1, \ldots, X_n$ drawn from a single Bernoulli distribution with parameter θ. In other words, for each X_i, we know that

$$P(X_i = 1) = \theta \quad \text{and} \quad P(X_i = 0) = 1 - \theta.$$

Our goal is to estimate the value of θ from these observed values of X_1 through X_n.
For any hypothetical value $\hat{\theta}$, we can compute the probability of observing the outcome $X_1, \ldots, X_n$ if the true parameter value θ were equal to $\hat{\theta}$. This probability of the observed data is often called the *data likelihood*, and the function $L(\hat{\theta}) = P(X_1, \ldots, X_n|\hat{\theta})$ that maps each $\hat{\theta}$ to the corresponding likelihood is called the *likelihood function*. A natural way to estimate the unknown parameter θ is to choose the $\hat{\theta}$ that maximizes the likelihood function. Formally,

$$\hat{\theta}^{\text{MLE}} = \arg\max_{\hat{\theta}} L(\hat{\theta}).$$

Often it is more convenient to work with the log likelihood function $\ell(\hat{\theta}) = \log L(\hat{\theta})$. Since the log function is increasing, we also have

$$\hat{\theta}^{\text{MLE}} = \arg\max_{\hat{\theta}} \ell(\hat{\theta}).$$

2.1 Bernoulli likelihood [5 points]

Write a formula for the log likelihood function, $\ell(\hat{\theta})$. Your function should depend on $N_1 = \#\{X_i = 1\}$ and $N_0 = \#\{X_i = 0\}$ and the hypothetical parameter $\hat{\theta}$, and should be simplified as far as possible (i.e. don't just write the definition of the log likelihood function).
The likelihood function for n i.i.d. samples from a Bernoulli distribution with parameter $\hat{\theta}$ is

$$L(\hat{\theta}) = \prod_{i=1}^{n} P(X_i \mid \hat{\theta}) = \hat{\theta}^{N_1}(1-\hat{\theta})^{N_0}.$$

Taking the natural logarithm, the log likelihood function simplifies to

$$\ell(\hat{\theta}) = \log L(\hat{\theta}) = N_1 \log \hat{\theta} + N_0 \log(1-\hat{\theta}).$$

2.2 Bernoulli example [5 points]

Consider the following sequence of 10 samples:

$$X = (0, 1, 0, 1, 1, 0, 0, 1, 1, 1).$$

Compute the maximum likelihood estimate for the 10 samples. Show all of your work (hint: recall that if x^* maximizes $f(x)$, then $f'(x^*) = 0$).
Let $k = \#\{X_i = 1\} = 6$ and $n = 10$. The Bernoulli likelihood function is

$$L(\hat{\theta}; X) = \hat{\theta}^k (1-\hat{\theta})^{n-k}.$$

Differentiating with respect to $\hat{\theta}$ and setting the derivative to zero gives:

$$\frac{d}{d\hat{\theta}} L(\hat{\theta}) = \hat{\theta}^{k-1}(1-\hat{\theta})^{n-k-1}(k - n\hat{\theta}) = 0.$$

Solving for $\hat{\theta}$:

$$\boxed{\hat{\theta} = \frac{k}{n}}.$$

Alternatively, using the log-likelihood

$$\log L(\hat{\theta}) = k \log \hat{\theta} + (n-k)\log(1-\hat{\theta}),$$

[2] independent & identically distributed.

we take the derivative and set it to zero:

$$\frac{k}{\hat{\theta}} - \frac{n-k}{1-\hat{\theta}} = 0 \quad \Rightarrow \quad \boxed{\hat{\theta} = \frac{k}{n} = \frac{6}{10} = 0.6.}$$

2.3 Binomial likelihood [5 points]

Now we will consider a related distribution. Suppose we observe the values of m iid random variables $Y_1, \ldots, Y_m$ drawn from a single Binomial distribution $B(n, \theta)$. A Binomial distribution models the number of 1's from a sequence of n independent Bernoulli variables with parameter θ. In other words,

$$P(Y_i = k) = \binom{n}{k}\theta^k(1-\theta)^{n-k} = \frac{n!}{k!(n-k)!} \cdot \theta^k(1-\theta)^{n-k}.$$

Write a formula for the log likelihood function, $\ell(\hat{\theta})$. Your function should depend on $S = \sum_{i=1}^m Y_i$ and $T = \sum_{i=1}^m \log \binom{n}{Y_i}$ and the hypothetical parameter $\hat{\theta}$.

For $Y_i \overset{iid}{\sim} \text{Binomial}(n, \hat{\theta})$,

$$P(Y_i = y_i) = \binom{n}{y_i}\hat{\theta}^{\,y_i}(1-\hat{\theta})^{\,n-y_i}.$$

Hence the log-likelihood over m samples is

$$\ell(\hat{\theta}) = \sum_{i=1}^m \log\binom{n}{Y_i} + \Big(\sum_{i=1}^m Y_i\Big)\log\hat{\theta} + \Big(mn - \sum_{i=1}^m Y_i\Big)\log(1-\hat{\theta}).$$

Defining $S = \sum_{i=1}^m Y_i$ and $T = \sum_{i=1}^m \log \binom{n}{Y_i}$, this simplifies to

$$\boxed{\ell(\hat{\theta}) = T + S\log\hat{\theta} + (mn - S)\log(1-\hat{\theta})}.$$

2.4 Binomial example [5 points]

Consider two Binomial random variables Y_1 and Y_2 with the same parameters, $n = 5$ and θ. The Bernoulli variables for Y_1 and Y_2 resulted in $(0, 1, 0, 1, 1)$ and $(0, 0, 1, 1, 1)$, respectively. Therefore, $Y_1 = 3$ and $Y_2 = 3$. Compute the maximum likelihood estimate for the 2 samples. Show your work.

We have $m = 2$ i.i.d. Binomial$(n = 5, \hat{\theta})$ samples with $Y_1 = 3$ and $Y_2 = 3$. Let $S = \sum_{i=1}^2 Y_i = 6$ and $mn = 2 \cdot 5 = 10$. The binomial log-likelihood is

$$\ell(\hat{\theta}) = T + S\log\hat{\theta} + (mn - S)\log(1-\hat{\theta}),$$

so

$$\frac{d\ell}{d\hat{\theta}} = \frac{S}{\hat{\theta}} - \frac{mn-S}{1-\hat{\theta}} = 0 \implies \hat{\theta} = \frac{S}{mn} = \frac{6}{10} = 0.6.$$

Therefore, $\boxed{\hat{\theta}_{\text{MLE}} = 0.6}$.

2.5 Comparison [5 points]

How do your answers for parts 1 and 3 compare? What about parts 2 and 4? If you got the same or different answers, why was that the case?

Parts 1 vs. 3: Comparison of log-likelihood formulas

In Part 1 (Bernoulli), we derived the log-likelihood as:

$$\ell(\hat{\theta}) = N_1\log\hat{\theta} + N_0\log(1-\hat{\theta})$$

In Part 3 (Binomial), we derived:

$$\ell(\hat{\theta}) = T + S\log\hat{\theta} + (mn - S)\log(1-\hat{\theta})$$

where

$$S = \sum_{i=1}^{m} Y_i, \quad T = \sum_{i=1}^{m} \log \binom{n}{Y_i}$$

If we set $n = 1$, the binomial model reduces to the Bernoulli case. Since $\binom{1}{Y_i} = 1$, we have $T = 0$, $S = N_1$, and $mn = m$. Thus, the binomial log-likelihood simplifies exactly to the Bernoulli formula.

$\Rightarrow$ Conclusion: Part 3 is a more general form, and Part 1 is a special case of it.

Parts 2 vs. 4: Comparison of MLE estimates

In Part 2, we found the MLE for 10 Bernoulli samples with 6 successes:

$$\hat{\theta} = \frac{6}{10} = 0.6$$

In Part 4, we considered two binomial samples each with $n = 5$, and the total number of successes was again $S = 6$ out of $mn = 10$ trials. The MLE was:

$$\hat{\theta} = \frac{S}{mn} = \frac{6}{10} = 0.6$$

The estimates are identical because the MLE for θ in both Bernoulli and Binomial models depends only on the total number of successes S and total number of trials mn, not on how the data are grouped. The binomial coefficient term does not depend on θ and thus vanishes during differentiation.

$\Rightarrow$ Conclusion: The MLE is the same regardless of whether the data is treated as Bernoulli or Binomial.

3 Logistic regression

In this question, we will study the logistic regression algorithm.

3.1 Conditional likelihood [10 points]

For simplicity, we assume the dataset is two-dimensional. Given a training set $\{(x^i, y^i), i = 1, \ldots, n\}$ where $x^i \in \mathbb{R}^2$ is a feature vector and $y^i \in \{0, 1\}$ is a binary label, we want to find the parameters $\hat{w}$ that maximize the likelihood for the training set, assuming a parametric model of the form

$$p(y = 1|x; w) = \frac{1}{1 + \exp(-w_0 - w_1 x_1 - w_2 x_2)} = \frac{\exp(w_0 + w_1 x_1 + w_2 x_2)}{1 + \exp(w_0 + w_1 x_1 + w_2 x_2)}. \tag{1}$$

Below, we give a derivation of the conditional log likelihood. In this derivation, **provide a short justification for why each line follows from the previous one**.

$$\ell(w) \equiv \ln \prod_{j=1}^{n} p(y^j \mid x^j, w) \tag{2}$$

$$= \sum_{j=1}^{n} \ln p(y^j \mid x^j, w) \quad \text{(Logarithm identity: } \ln(\prod_{j=1}^{n} a_j) = \sum_{j=1}^{n} \ln a_j) \tag{3}$$

$$= \sum_{j=1}^{n} \ln \left(p(y^j = 1 \mid x^j, w)^{y^j} p(y^j = 0 \mid x^j, w)^{1-y^j} \right) \tag{4}$$

(Bernoulli PMF: since $y^j \in \{0, 1\}$, $p(y^j|x^j, w) = p_1^{y^j} p_0^{1-y^j}$)

$$= \sum_{j=1}^{n} \left[y^j \ln p(y^j = 1 \mid x^j, w) + (1 - y^j) \ln p(y^j = 0 \mid x^j, w) \right] \tag{5}$$

(Property of logarithms: $\ln(a^s b^t) = s \ln a + t \ln b$)

$$= \sum_{j=1}^{n} \left[y^j \ln \frac{\exp(w_0 + w_1 x_1^j + w_2 x_2^j)}{1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j)} + (1 - y^j) \ln \frac{1}{1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j)} \right] \tag{6}$$

(Substitute the logistic model: $p(y = 1|x^j, w) = \frac{e^{w_0 + w_1 x_1^j + w_2 x_2^j}}{1 + e^{w_0 + w_1 x_1^j + w_2 x_2^j}}$ and $p(y = 0|x^j, w) = \frac{1}{1 + e^{w_0 + w_1 x_1^j + w_2 x_2^j}}$)

$$= \sum_{j=1}^{n} \left[y^j \ln \left(\exp(w_0 + w_1 x_1^j + w_2 x_2^j) \right) + \ln \left(\frac{1}{1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j)} \right) \right] \tag{7}$$

(Use $\ln\left(\frac{a}{b}\right) = \ln a - \ln b$ and simplify terms using $y^j + (1 - y^j) = 1$)

$$= \sum_{j=1}^{n} \left[y^j \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) - \ln \left(1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j) \right) \right]. \tag{8}$$

(Property of logarithms: $\ln(e^a) = a$ and $\ln\left(\frac{1}{1 + e^a}\right) = -\ln(1 + e^a)$)

3.2 Deriving the gradient [10 points]

Next, we will derive the gradient of the previous expression with respect to w_0, w_1, w_2, i.e., $\frac{\partial \ell(w)}{\partial w_i}$, where $\ell(w)$ denotes the log likelihood from part 1. We will perform a few steps of the derivation, and then ask you to do one step at the end. If we take the derivative of Expression 8 with respect to w_i for $i \in \{1, 2\}$, we get the following expression:

$$\frac{\partial \ell(w)}{\partial w_i} = \frac{\partial}{\partial w_i} \sum_{j=1}^{n} \left[y^j \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right] - \frac{\partial}{\partial w_i} \sum_{j=1}^{n} \ln \left[1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right]. \tag{9}$$

The blue expression is linear in w_i, so it can be simplified to $\sum_{j=1}^{n} y^j x_i^j$. For the red expression, we use the chain rule as follows (first we consider a single $j \in [1, n]$)

$$\frac{\partial}{\partial w_i} \ln \left[1 + \exp\left(w_0 + w_1 x_1^j + w_2 x_2^j\right)\right] \tag{10}$$

$$= \frac{1}{1 + \exp\left(w_0 + w_1 x_1^j + w_2 x_2^j\right)} \cdot \frac{\partial}{\partial w_i}\left(1 + \exp\left(w_0 + w_1 x_1^j + w_2 x_2^j\right)\right) \tag{11}$$

$$= \frac{1}{1 + \exp\left(w_0 + w_1 x_1^j + w_2 x_2^j\right)} \cdot \exp\left(w_0 + w_1 x_1^j + w_2 x_2^j\right) \frac{\partial}{\partial w_i}\left(w_0 + w_1 x_1^j + w_2 x_2^j\right) \tag{12}$$

$$= x_i^j \cdot \frac{\exp\left(w_0 + w_1 x_1^j + w_2 x_2^j\right)}{1 + \exp\left(w_0 + w_1 x_1^j + w_2 x_2^j\right)} \tag{13}$$

Now use Equation 13 (and the previous discussion) to show that overall, Expression 9, i.e., $\frac{\partial \ell(w)}{\partial w_i}$, is equal to

$$\frac{\partial \ell(w)}{\partial w_i} = \sum_{j=1}^{n} x_i^j (y^j - p(y^j = 1 \mid x^j; w)) \tag{14}$$

Hint: does Expression 13 look like a familiar probability?
Using (9) and (13),

$$\frac{\partial \ell(w)}{\partial w_i} = \sum_{j=1}^{n} y^j x_i^j - \sum_{j=1}^{n} x_i^j \frac{\exp(w_0 + w_1 x_1^j + w_2 x_2^j)}{1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j)} = \sum_{j=1}^{n} x_i^j \left(y^j - p(y^j = 1 \mid x^j; w)\right).$$

Since the log likelihood is concave, it is easy to optimize using gradient ascent. The final algorithm is as follows. We pick a step size η, and then perform the following iterations until the change is $< \epsilon$:

$$w_0^{(t+1)} = w_0^{(t)} + \eta \sum_j \left[y^j - p(y^j = 1 \mid x^j; w^{(t)})\right] \tag{15}$$

$$w_i^{(t+1)} = w_i^{(t)} + \eta \sum_j x_i^j \left[y^j - p(y^j = 1 \mid x^j; w^{(t)})\right]. \tag{16}$$

3.3 The decision boundary [5 points]

Recall the prediction rule for logistic regression is if $p(y^j = 1 \mid x^j) > p(y^j = 0 \mid x^j)$, then predict 1, otherwise predict 0. Prove that this is a linear decision boundary using only the prediction rule and the parametric form of the probability.
We want to prove that logistic regression produces a linear decision boundary.
Step 1: Start from the prediction rule. The model predicts class 1 if:

$$p(y = 1 \mid x) > p(y = 0 \mid x)$$

Since $p(y = 0 \mid x) = 1 - p(y = 1 \mid x)$, this is equivalent to:

$$p(y = 1 \mid x) > 0.5$$

Step 2: Substitute the logistic function. From the logistic regression model:

$$p(y = 1 \mid x) = \frac{1}{1 + \exp(-w^T x)}$$

where $w^T x = w_0 + w_1 x_1 + w_2 x_2$. Thus, the inequality becomes:

$$\frac{1}{1 + \exp(-w^T x)} > 0.5$$

Step 3: Solve the inequality. Multiply both sides by $1 + \exp(-w^T x)$, which is always positive:

$$1 > 0.5\,(1 + \exp(-w^T x))$$

$$0.5 > 0.5\exp(-w^T x)$$

$$1 > \exp(-w^T x)$$

Step 4: Take the natural logarithm. Since $\ln$ is strictly increasing:

$$\ln(1) > \ln(\exp(-w^T x)) \implies 0 > -w^T x$$

$$w^T x > 0$$

Step 5: Interpret the result. The classifier predicts:

$$\hat{y} = \begin{cases} 1, & \text{if } w^T x > 0 \\ 0, & \text{if } w^T x \leq 0 \end{cases}$$

The decision boundary occurs where $w^T x = 0$, i.e.,

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

which is the equation of a straight line in 2D (or a hyperplane in higher dimensions). Hence, logistic regression yields a **linear decision boundary**.